

Protoreasoning in Tiny Transformers

Eduardo Valle *Fin AI Research*
Fergal Reid *Fin AI Research*

Abstract

We show that tiny transformers can profitably employ a simple form of chain of thought, which we call protoreasoning, allowing us to study step-by-step reasoning on $\sim 1\text{M}$ -parameter models and opening up opportunities for much more detailed experimentation and analysis than is feasible for larger models. Current large language models exhibit impressive step-by-step reasoning, but we have yet to understand its generality, i.e., when and how LLMs learn genuinely general algorithms rather than “bags of heuristics.” Such questions are hard to settle on compute-intensive frontier models trained on opaque data. To work at model scales far below the threshold for natural-language competence, we define reasoning-friendly tasks on Dyck languages (sentences of correctly nested brackets). We find that protoreasoning traces substantially close the out-of-distribution generalization gap, and ablations confirm that the trace’s content, not merely its extra tokens, drives the gain.

1 Introduction

Chain of thought (CoT) reasoning is crucial for getting the best out of large language models (LLMs), yet hard to study on frontier models too costly to train repeatedly. Small transformers are far more wieldy for comprehensive experimentation and interpretability, but sit orders of magnitude below the scale at which natural-language reasoning becomes viable. We bridge that gap, inducing tiny models ($\sim 1\text{M}$ parameters) to reason in a primitive but useful way.

LLMs have demonstrated remarkable step-by-step reasoning capabilities for tasks ranging from mathematical word problems to coding, spurring intense research into understanding and improving procedural reasoning, i.e., the ability to solve problems systematically by following sound procedures. Inference-time techniques such as CoT consistently enhance accuracy on mathematical, logical, and code reasoning tasks (Wei et al., 2022b). However, current frontier models fail on sufficiently complex instances of computational tasks (Shojaee et al., 2025), suggesting they never learn truly generalizable algorithms. Whether this reflects inherent architectural limitations or practical constraints of context length and compute budget is still contested (Lawsen, 2025; Khan et al., 2025; Varela et al., 2025), but mechanistic evidence also suggests that models achieve algorithmic behavior through ad-hoc “bags of heuristics”, i.e., collections of narrow rules each firing on specific inputs (Nikankin et al., 2025). Other works suggest that much of current CoT amounts to post hoc justification rather than a priori deliberation (Turpin et al., 2023; Lanham et al., 2023). Settling those and other disputes on frontier LLMs is hard due to opaque training data and the sheer computational cost of training them.

We sidestep those obstacles by moving to the opposite end of the scale. Tiny transformers have proven useful “model organisms” for isolating mechanisms that frontier models hopelessly entangle (Nanda et al., 2023). Formal languages supply the graded, verifiable challenge such models need: because we can easily verify membership in simple formal languages and control their structure, we can generate inputs of any desired difficulty and verify results exactly (Strobl et al., 2024; Vafa et al., 2024). Concretely, we build two tasks over the natural bijection between forests of trees and Dyck- k languages (sentences with up to k bracket-

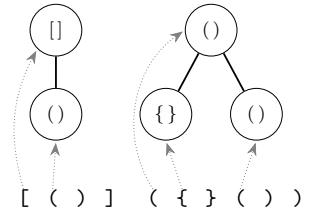


Figure 1: A Dyck sentence is formed by well-balanced brackets and corresponds to a forest of trees, with inner brackets descending from outer ones. Dyck languages are a central object in theoretical linguistics for their ability to represent arbitrary depths of recursion.

types, correctly nested, see Fig. 1). The tasks admit protoreasoning traces: deterministic scratchpads that repeatedly prune tree nodes that cannot belong to the answer.

Our contributions are:

- We induce a primitive but useful form of step-by-step reasoning in models far below the parameter scale required for natural-language competence, which can serve as minimal systems for studying reasoning.
- Based on the bijection between Dyck- k languages and forests of trees, we propose two reasoning-friendly tasks with gradable difficulty and exact verifiability.
- We leverage the small size of our models to perform dense experimental designs, comprising over 1700 training runs.

2 Methodology

2.1 Tasks

In order to allow our models to display something akin to reasoning, we had to design tasks that were simple enough to be feasible given their small size, but challenging enough to benefit from step-by-step progression. Traditional, natural-language CoT was out of the question, for our models are much below the parameter count of even first-generation transformers (Vaswani et al., 2017). Instead, we focused on simple formal languages and a latent syntax–semantics correspondence, then defined a couple of tasks, each with output given by a deterministic function of the input. The challenge we pose to the models is not sampling strings from the formal language per se, but providing the single correct answer to each input prompt belonging to the language.

Dyck- k languages are the sets of all sentences with k types of brackets correctly nested and balanced. For example, $([])$ is a Dyck- k sentence for $k \geq 2$, but $[()]$ is not a Dyck sentence for any k . Theoretical linguistics has studied these languages extensively for their ability to represent arbitrary depths of grammatical recursion while remaining extremely simple.

Semantically, each Dyck sentence can be interpreted as a tree (or forest), with inner brackets representing descendants of embracing outer brackets. Using that correspondence, we define two tasks: deepest path (DP), which outputs the opening brackets along the path that leads to the deepest leaf in the sentence, and maximum-order preleaf (PL), which outputs the opening brackets of the biggest group of siblings composed entirely of leaves. In both tasks the expected answer holds only the *opening* brackets. Ties are broken in favor of the rightmost candidate (Fig. 2).

We characterize each sentence by its **half-length** hl (number of bracket pairs in the sentence) and a **structural parameter** related to the task (depth or maximum preleaf order). Then, we explore generalization across those two axes.

Why tiny models? Scale is crucial for LLM performance: arguably, the transformer decoder became the dominant architecture for language models because it scales up so well. Still, tiny LLMs have proven useful “model organisms” for exploring phenomena that appear in much bigger networks. State-of-the-art LLMs take weeks and require dozens of multi-GPU nodes to train. “Small” models (1 to 10B parameters) still can take days on an expensive node to pretrain. In contrast, our entire study, with thousands of training runs, takes little more than a day on a multi-GPU node. See Sec. A.4 for details on compute.

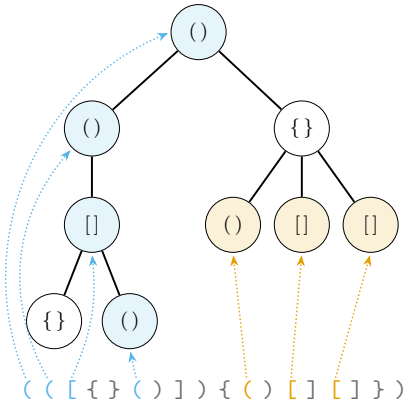


Figure 2: Tasks. **Deepest path** (DP, answer in blue): path to the deepest leaf. **Maximum-order preleaf** (PL, answer in orange): largest all-leaf sibling brood. Ties are broken rightmost and the answer has only the opening brackets. Input Dyck sentence at the figure’s bottom.

<pre> <BOS> ((({ } ())) { () [] [] }) <BOT> ((({ } ())) { [] [] }) ← ((({ } ())) { [] }) ← ((({ } ())) { }) ← ((({ } ()))) ← ((([]))) ← <EOT> ((((<EOS> </pre>	<pre> <BOS> ((({ } ())) { () [] [] }) ! <BOT> (({ } ())) { () [] [] } ← [{ } ()] { () [] [] } ← { () [] [] } ← <EOT> ([<EOS> </pre>
--	--

Figure 3: Training samples for input and tasks of Fig. 2. A token separates input and expected output: | for DP (left) and ! for PL (right). Each step of the reasoning trace prunes part of the trees until only the answer remains. The $\leftarrow$ marks newline tokens between steps: all other spacing is just for visualization. Special <tokens> delimit the sentence and the trace. Vanilla samples omit the trace and its delimiters.

2.2 Protoreasoning traces

The main experimental intervention is adding an explicit reasoning trace to training samples, which solves the task step-by-step by eliminating brackets/tree nodes that cannot be part of the solution. Thus, we propose every task in two formats: a **vanilla** training sample has, in sequence, the input sentence, the task separator, and the expected output; a **trace** training sample adds a reasoning trace $s_1, \dots, s_m$ (Fig. 3). Each step s_i is the previous step s_{i-1} after pruning some part of the tree that cannot be part of the solution.

For DP, at each step, we eliminate the shallowest leaf still present, until the tree degenerates into a path graph. For PL, we peel the trees by the root, and whenever two or more trees have only two levels, we eliminate the smallest one, until only a single tree remains.¹ In both cases, if there are tied candidates for elimination, we prune the leftmost.

To make the traces more challenging and to stimulate generalization, we introduce a **step dropout**, which randomly skips reasoning steps.

The traces are never part of the prompts we use for evaluation. At test time, both formats are prompted with the sample prefix up to and including the task separator.

2.3 Experimental setting

Unless explicitly stated otherwise, we use:

- Dyck-4 languages with sentences of 32 bracket pairs (half-length $hl = 32$).
- A miniaturized Llama 2 model with 4 layers, residual width 128, 16 heads, and no grouped-query attention.
- 10 random seeds affecting model-weight initialization and training-data sampling.
- An 8-trial quasi-random hyperparameter search per treatment. For each treatment, hyperparameter search runs only once (for the first random seed).
- A positional-encoding data augmentation (SkipAlign, see below).
- For trace treatments, step dropout with a drop probability of 15%, which we apply independently for each reasoning step.

¹As a special case, if the input is a forest of single nodes (e.g., $\{\}\{\}\{\}$), we consider that forest the largest sibling brood.

Because we are operating close to the limit of the capacity of the models (see Sec. A.4), the experiments can be noisy, with models sometimes alternating between total failure and perfect success depending on how we initialize weights and data sampling. To cope with such noise, we average all reported numbers over the 10 random seeds.

Hold-out patterns. Each value of the structural parameter is a data **stratum** in the design. We probe generalization by splitting the strata into disjoint training, validation, and test sets, using six different patterns, named by which strata are held out (validation+test): **R**andom, two **B**locks, **E**ven, **M**iddle, **H**igh, and **L**ow. R/B/E/M emphasize interpolation (short-range for R/E, mid-range for B, long-range for M), while H/L emphasize extrapolation. The exact splits we employed appear in Fig. 7 in the Appendix, and are also visible in the styling of the x-axes’ ticks in Fig. 5 and Fig. 6.

Metrics. The main metric is output **validity**: whether the output is correct given the input. We take as output anything after the `<EOT>` token (or the task separator, if `<EOT>` is not present) and before the `<EOS>` token. At every step, we draw fresh sentences from a generator so large that, for all practical purposes, we may consider that the model never trains on the same sentence twice. Thus, the validity on each new training batch already measures in-domain generalization (new sentences on already seen train strata). However, unless we explicitly state otherwise, we measure and report **out-of-distribution/OOD generalization** averaged over the unseen test strata.

Deconfounding positional-encoding generalization. The length of any sample depends on both the half-length and structural parameter of the input, but the range of lengths is much wider for trace samples. Using standard rotary positional encoding (RoPE), trace pays a big penalty because we test it on positional encodings that differ substantially from those it saw in training. To eliminate this confounder, we train all models with **SkipAlign** (Wu et al., 2024), a PoSE-style (Zhu et al., 2024) positional augmentation that inserts position-index gaps at strategic boundaries. SkipAlign is a form of data augmentation that allows training the model with much more diverse positional encodings, without paying the quadratic cost of using large contexts filled with actual data.

More details, including hyperparameter search, training, and data sampling, appear in the Appendix.

3 Experiments Design and Results

We summarize our main findings below. The corresponding subsections report details and further analyses.

The proposed tasks are easy to express but hard to generalize. Our tasks strike the right level of challenge for the models we evaluate (Sec. 3.1). A model trained on a single stratum reaches near-perfect validity when tested in-distribution, but collapses out-of-distribution, especially along the structural axis (Tab. 1).

Protoreasoning traces improve generalization. When we instead train on diverse strata, out-of-distribution generalization markedly improves. Across two tasks and six hold-out patterns, the trace format almost always beats the vanilla one (Sec. 3.2, Tab. 2).

Protoreasoning traces help mixed tasks. The advantage of protoreasoning traces increases when we train a single model on both tasks at once: trace tolerates the multi-task setting far better than vanilla (Sec. 3.3, Tab. 3).

Trace’s improvement stems from its contents, not simply the extra tokens it adds. Replacing trace content with filler dots collapses the gains (Tab. 4, Sec. 3.4).

3.1 The proposed tasks are easy to express but hard to generalize

All experiments in this section use the vanilla sample format, and vary half-lengths and structural parameters.

Table 1: Expressivity versus generalization for models trained on a single stratum of the vanilla format. Output validity (%) averaged over the concerned strata and 10 random seeds $\pm$ standard deviation over the seeds, for three training half-lengths (hl). In-domain validity is near perfect, but generalization crashes out-of-distribution, especially along the structural axis (depth/max. preleaf).

TRAIN HL	TRAIN STRUCT.	IN-DOMAIN	HL OOD	STRUCT. OOD	BOTH OOD
DEEPEST PATH					
32	16	99.8 ± 0.5	34.9 ± 13.5	0.0 ± 0.0	0.2 ± 0.2
64	32	99.8 ± 0.4	4.4 ± 4.7	0.0 ± 0.0	0.3 ± 0.4
128	64	98.4 ± 1.8	1.8 ± 1.4	0.0 ± 0.0	0.0 ± 0.0
MAX. PRELEAF					
32	8	99.7 ± 0.4	62.6 ± 28.2	0.0 ± 0.0	0.5 ± 0.9
64	16	99.8 ± 0.3	91.1 ± 10.3	0.2 ± 0.5	0.6 ± 1.5
128	32	99.7 ± 0.4	50.1 ± 7.4	0.0 ± 0.0	0.0 ± 0.0

Table 2: Generalization across different holdout patterns for models trained and evaluated on a single task (deepest path/DP or maximum-order preleaf/PL) and format (vanilla or trace). Output validity averaged over the test strata and 10 random seeds $\pm$ standard deviation over the seeds. Reasoning traces almost always improve generalization.

PATTERN	DEEPEST PATH		MAX. PRELEAF	
	VANILLA	TRACE	VANILLA	TRACE
Random	93.6 ± 4.5	95.4 ± 2.1	91.7 ± 2.4	97.8 ± 0.5
Blocks	94.6 ± 2.4	98.7 ± 0.4	76.9 ± 7.0	97.3 ± 0.5
Even	98.5 ± 2.5	98.8 ± 0.9	88.4 ± 0.8	97.7 ± 0.5
Middle	80.5 ± 9.7	98.0 ± 2.0	85.8 ± 11.0	97.1 ± 1.1
High	50.9 ± 5.4	66.7 ± 9.4	23.9 ± 24.3	62.2 ± 7.3
Low	39.6 ± 2.0	21.6 ± 7.4	43.9 ± 3.0	41.0 ± 5.4

Table 3: Generalization across different holdout patterns for models trained on both tasks (DP and PL) and evaluated on each of them. Output validity averaged over the test strata and 10 random seeds $\pm$ standard deviation over the seeds. In general, trace tolerates multi-task training much better than vanilla.

PATTERN	EVALUATED ON DP		EVALUATED ON PL	
	VANILLA	TRACE	VANILLA	TRACE
Even Mixed	62.7 ± 33.2	98.6 ± 0.2	74.4 ± 15.8	94.5 ± 1.6
Middle Mixed	80.7 ± 16.8	92.7 ± 2.9	60.5 ± 24.1	94.8 ± 1.7
High Mixed	46.6 ± 5.2	43.0 ± 4.9	17.7 ± 13.7	60.5 ± 7.7

Table 4: Ablations across holdout patterns. Output validity averaged over the test strata and 10 random seeds $\pm$ standard deviation over the seeds. Both data augmentations (SkipAlign and step dropout) help improve generalization, especially on deepest path, which has longer reasoning traces. Erasing the trace content with dots dramatically deteriorates results, showing that increasing the context size does not suffice to explain trace’s contribution.

PATTERN	TRACE					VANILLA	
	ALL STEPS		+ STEP DROPOUT		ERASED	RoPE	SKIPALIGN
	RoPE	SKIPALIGN	RoPE	SKIPALIGN			
DEEPEST PATH							
Random	59.8 ± 24.4	85.2 ± 8.5	94.4 ± 1.7	95.4 ± 2.1	9.8 ± 9.0	88.9 ± 3.8	93.6 ± 4.5
Blocks	81.1 ± 15.2	94.3 ± 7.6	99.2 ± 0.5	98.7 ± 0.4	5.4 ± 9.0	94.9 ± 2.5	94.6 ± 2.4
Even	64.6 ± 38.7	91.7 ± 20.1	99.1 ± 0.3	98.8 ± 0.9	0.5 ± 0.8	95.7 ± 10.6	98.5 ± 2.5
Middle	87.8 ± 23.0	93.4 ± 11.4	87.5 ± 10.2	98.0 ± 2.0	4.3 ± 6.7	76.3 ± 5.9	80.5 ± 9.7
High	43.5 ± 16.0	54.7 ± 9.6	53.5 ± 3.0	66.7 ± 9.4	0.0 ± 0.0	49.3 ± 3.7	50.9 ± 5.4
Low	3.5 ± 3.7	18.7 ± 11.9	10.9 ± 8.1	21.6 ± 7.4	0.0 ± 0.0	34.4 ± 3.6	39.6 ± 2.0
MAX. PRELEAF							
Random	98.1 ± 1.9	98.7 ± 0.9	97.8 ± 0.9	97.8 ± 0.5	30.3 ± 12.9	84.1 ± 6.2	91.7 ± 2.4
Blocks	97.5 ± 0.6	97.2 ± 0.9	97.1 ± 0.6	97.3 ± 0.5	44.5 ± 5.3	79.8 ± 10.5	76.9 ± 7.0
Even	95.8 ± 3.6	97.5 ± 3.0	96.4 ± 2.0	97.7 ± 0.5	36.4 ± 13.7	82.3 ± 12.8	88.4 ± 0.8
Middle	94.5 ± 3.8	97.5 ± 1.4	96.7 ± 1.8	97.1 ± 1.1	29.1 ± 14.3	79.9 ± 15.4	85.8 ± 11.0
High	48.9 ± 4.4	58.1 ± 9.6	56.6 ± 8.6	62.2 ± 7.3	0.0 ± 0.0	19.7 ± 16.7	23.9 ± 24.3
Low	35.7 ± 9.3	47.6 ± 3.9	34.6 ± 9.6	41.0 ± 5.4	11.3 ± 7.3	44.0 ± 3.4	43.9 ± 3.0

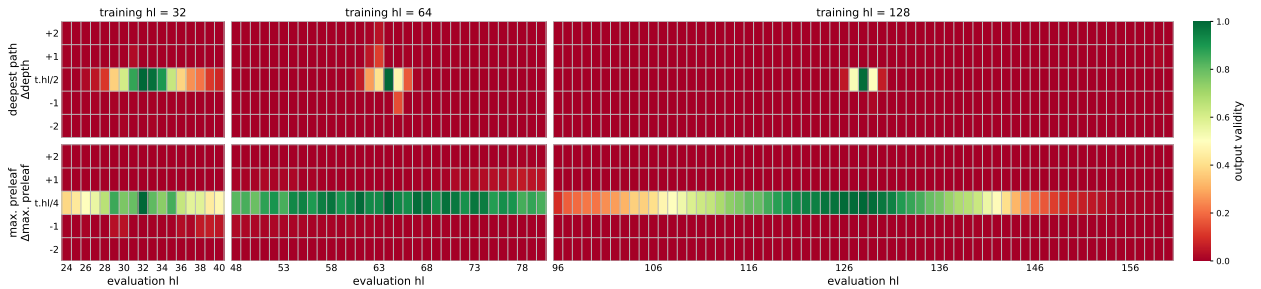


Figure 4: Six experiments showcasing the gap between expressivity and generalization for the tasks proposed. We train the models with a single stratum, with in-distribution validity appearing as the dark-green square at the center of each subplot, showing the models are capable of expressing the task. All other squares in the subplot are out-of-distribution (hl and structure offsets shown horizontally and vertically, respectively). Both length-wise and structure-wise generalizations are hard, but the latter is especially difficult.

Tab. 1 showcases the tension between expressivity and generalization. Near-perfect validity shows that the model has enough capacity to express the correct solution for in-distribution prompts, even for over a hundred brackets and very complex structure. However, the picture reverses out-of-distribution: validity is only partly retained along the length axis (hl), and all but vanishes along the structural axis (depth for DP, maximum preleaf for PL).

Fig. 4 shows the same results broken down by test stratum. The dark-green square in the middle of each sub-plot shows the in-distribution validity (always $\sim 100\%$). All other squares are out-of-distribution. Pure-length generalization strata are in the same row as the in-distribution square, while pure-structural generalization strata share the same column. It is immediately visible that the former are easier than the latter, although both remain challenging.

3.2 Protoreasoning traces help generalization, but extrapolation is still a challenge

The previous section showed that we should not expect zero-shot generalization when training with a single stratum, especially along the structural axis. In this section, we show that generalization is still possible if we train the model with a diverse array of strata. Thus, we split the strata into train, validation, and test using the holdout patterns explained in Sec. 2.3.

Tab. 2 shows that the trace format generalizes better than vanilla almost always across holdout patterns, often by a considerable margin. On the interpolation patterns (R, B, E, M) the trace format leads to validities above 95%, with small deviation across seeds. The extrapolation regime (H, L), however, is hard for both formats: validity drops precipitously and becomes much noisier. Pattern L is particularly challenging for trace, because it leads to holdout reasoning traces having many more steps than the ones seen in training.

Fig. 5, with results broken down by stratum, makes the generalization differences more evident. The in-domain strata used for training (straight x-axis ticks) have near-perfect validity, while the held-out out-of-domain squares (round ticks) may have anything from perfect to zero validity, depending on the hold-out pattern, data format, and specific stratum. In the interpolation regime (R, B, E, M), trace always beats vanilla, especially on the longer-range interpolation of B and M. Neither trace nor vanilla is able to extrapolate very far.

3.3 Protoreasoning traces help mixed tasks

Here, we train all models on both tasks (DP and PL) and then evaluate each task separately. We expect the models to provide the right output for the right task, using the input separators | or ! to identify whether the desired output is for DP or PL, respectively.

We stratify by the two structural parameters, depth and maximum pre-leaf, at once. Because the parameters interact (for a fixed number of nodes, a tree cannot, at once, be very deep and have a very large preleaf) we sample only from the combinations that have enough diversity so that the model does not just trivially memorize them. As usual, we sample a stratum uniformly at random, and then a sentence within the stratum uniformly at random, but now we have many more strata, reflecting the joint choice of parameters (Sec. A.1).

To evaluate out-of-distribution generalization, we adapt patterns E, M, and H to consider only the joint strata explained above. We apply the hold-out patterns independently for each task: DP samples apply the holdout pattern to depth, and PL samples to maximum preleaf. The independent stratification ensures that the model sees all possible input strata, but for each task separately, the train/test/validation split still applies.

Tab. 3 shows that mixed-task training is more challenging for both formats, but that validity drops much more for vanilla than for trace. Fig. 6 details this per stratum, where the gap is most visible on the longer-range interpolative patterns H and M. We hoped that some form of cross-transfer could happen, i.e., that one task would be able to better generalize to its held-out strata by noticing that those still appear for the other task, but that would have led to *improved* results in contrast to the pure tasks of Sec. 3.2, which was not the case. That suggests that generalization challenges are due to how the model wires the solution for each task, and not simply to the model being unable to represent a given input.

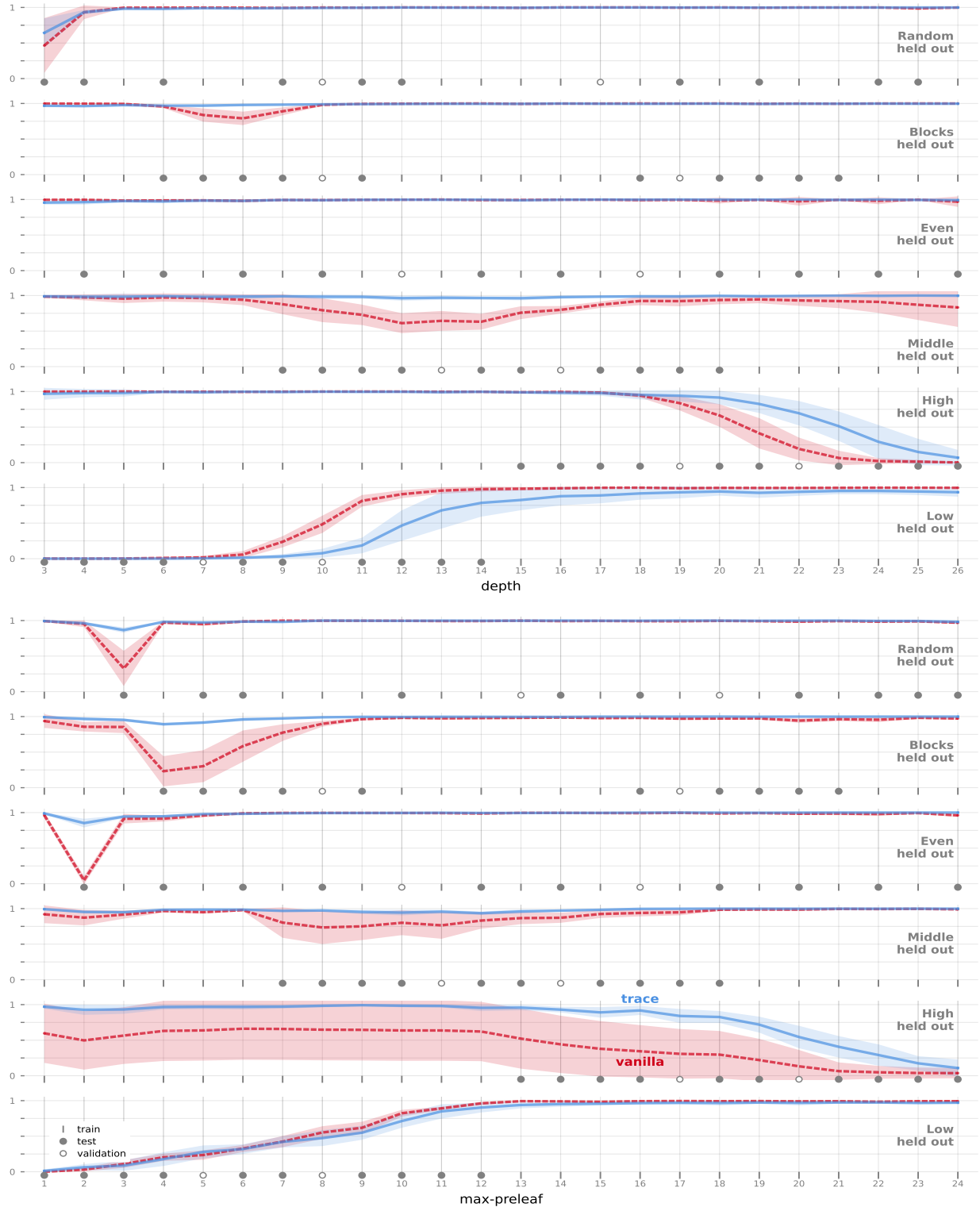


Figure 5: Same results as in Tab. 2, shown per stratum. Averaged output validities (lines) $\pm$ standard deviations (shaded areas) over 10 random seeds. Top: deepest path (DP), bottom: maximum-order preleaf (PL). Comparison between vanilla and trace formats for each pattern $\times$ task combination. Training, validation, and test strata are shown in the x-axes' tick shapes. In general, reasoning traces handle long interpolations and extrapolations better, although extrapolation remains hard for both formats.

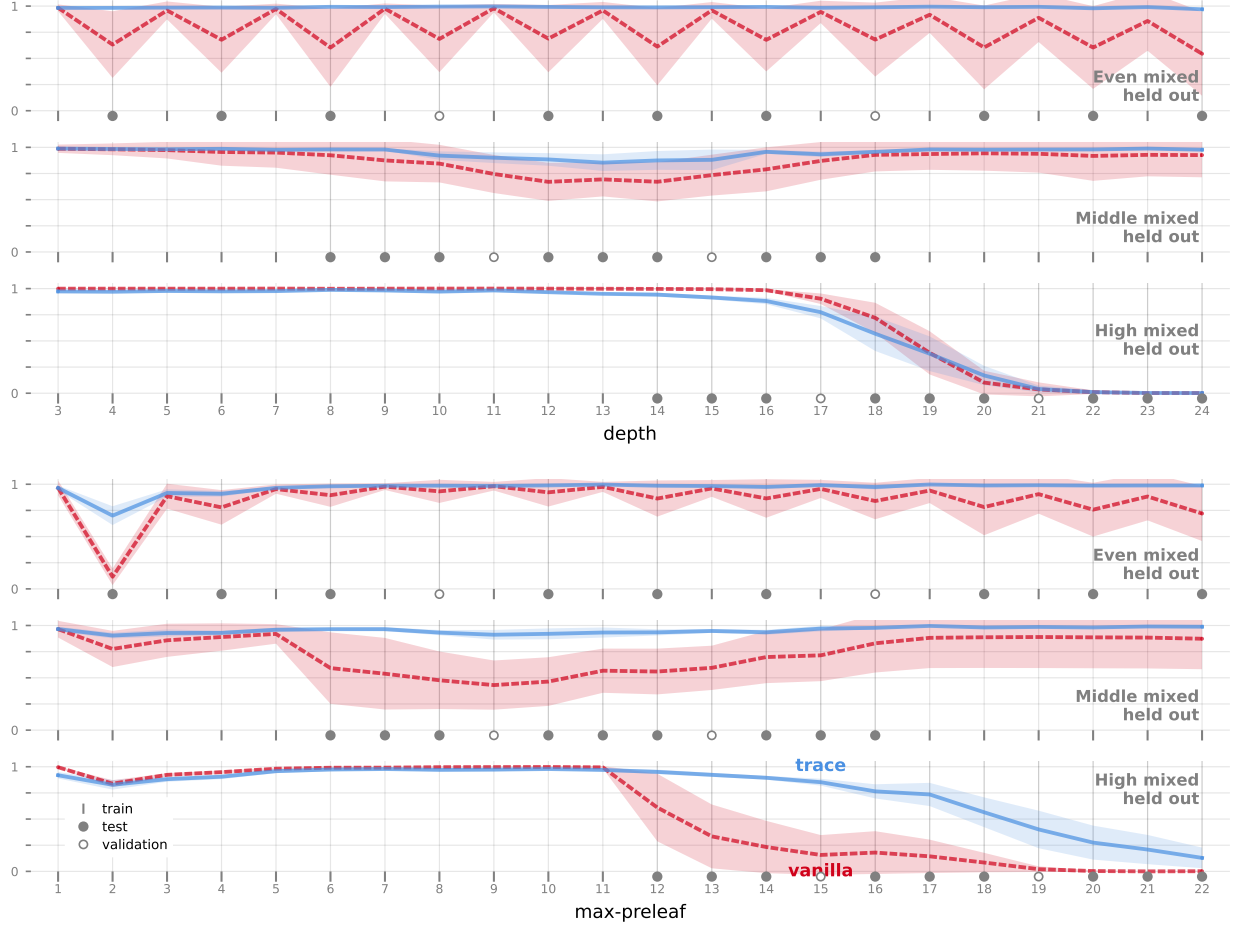


Figure 6: Same results as in Tab. 3, shown per stratum. Averaged output validities (lines) $\pm$ standard deviations (shaded areas) over 10 random seeds. All models trained on both tasks and evaluated separately on deepest path/DP (top) and maximum-order preleaf/PL (bottom). Training, validation, and test strata are shown in the x-axes' tick shapes. Trace preserves generalization better under mixed-task training than vanilla.

3.4 Ablations

To help the model generalize to new strata, we use two data augmentations. Both formats (vanilla and trace) employ SkipAlign, which introduces gaps on the index used to compute the positional encoding. Trace, additionally, employs step dropout, which randomly skips 15% of the reasoning steps.

Tab. 4 shows that SkipAlign improves results on both data formats and both tasks. Step dropout has a large positive effect on deepest path, which has long reasoning traces and a lot of length variability over strata. Although it slightly penalizes maximum preleaf, whose traces are much shorter, its average effect is still positive.

A possible confounder for the improvements brought by trace is the sheer addition of more tokens. Indeed, literature shows that additional non-informative tokens may improve performance simply by allowing the model a larger compute budget (Pfau et al., 2024). To exclude that possibility, we implement a “dot-by-dot” replacement of the reasoning traces at training time, which dramatically deteriorates validity, showing that the trace contents indeed contribute to the validity improvements.

4 Related Work

Step-by-step reasoning. Wei et al. (2022b) showed that simply prompting a large model to emit its intermediate steps, a technique that became known as CoT, markedly improves accuracy on arithmetic, commonsense, and symbolic reasoning, triggering a wave of work on step-by-step reasoning. A substantial survey literature now maps that landscape (Huang & Chang, 2023; Plaat et al., 2025; Chu et al., 2024; Sun et al., 2025; Zhang et al., 2025). Its recurring finding is that eliciting intermediate steps (CoT prompting, scratchpads, self-consistency, and tree or graph search) reliably improves accuracy on mathematical, logical, and code tasks, though the gains saturate as problem complexity grows. Tool integration (Schick et al., 2023; Yao et al., 2023b) and structured search (Yao et al., 2023a; Besta et al., 2024) further enhance and stabilize performance.

The limits of reasoning. Even with CoT, models remain brittle out-of-distribution (Mondorf & Plank, 2024). Shojaei et al. (2025) report that, for sufficiently complex instances of computational puzzles (Tower of Hanoi, river crossing, etc.), both reasoning and non-reasoning models eventually fail, with accuracy collapsing past a complexity threshold. Others contest whether this reflects an inherent limitation or an artifact of the evaluation: bounded token budgets, disallowed tools, unsatisfiable instances, or crediting only the strictly optimal solution (Lawsen, 2025; Khan et al., 2025; Varela et al., 2025). Another limitation is that reasoning traces are not necessarily faithful: the emitted steps need not reflect the computation that actually produced the answer (Turpin et al., 2023; Lanham et al., 2023).

Formal languages. Formal languages apply the rigorous lens of theoretical computer science to this investigation. The well-established Chomsky hierarchy of formal languages (regular, context-free, context-sensitive, and recursively enumerable), as well as a finer-grained web of languages characterized by the complexity class of the decision algorithm required to recognize them, allows us to gauge a model’s capability by asking which of those language classes it can generate or recognize.

Strobl et al. (2024) survey that literature, reporting bounds that place transformer-based models above (“at least”), below (“at most”), or in equivalence with complexity classes, and noting that the conclusions hinge on modeling assumptions (architecture, positional encoding, numerical precision, attention masking/activation) that often assume strong idealizations (infinite precision, hard attention). Under relatively realistic assumptions, some analyses (Chiang et al., 2023; Merrill & Sabharwal, 2023; Lin et al., 2021; Hahn, 2020) yield informative results: encoder-only models can express the languages in uniform-ACC⁰ under constant precision, extending to uniform-TC⁰ under $O(\log n)$ precision; decoder-only models are less well understood, but known counterexamples show they cannot recognize Parity or Dyck-2. These guarantees, however, speak to expressivity rather than learnability, i.e., whether a model can represent a language, not whether it will learn it from data.

Such limitations, which also include a focus on asymptotic analysis, motivate a more empirical approach, which employs real-world architectures without extra assumptions and measures learning from data. Vafa et al. (2024) measure, empirically, how GPT-2-like models learn a regular automaton (conceptualized as a world model), showing that next-token accuracy is a poor proxy for learning the true automaton: the LLM may display excellent next-token prediction and still break down completely under small deviations, demonstrating that a robust model of the automaton was not learned.

Allen-Zhu & Li (2025) pre-train GPT-2-style decoders on synthetic context-free grammars (CFGs) conceived to be long and locally ambiguous, so that deciding membership requires global disambiguation, finding that the models generate valid outputs and encode in their hidden states the grammar’s nonterminals, while their attention patterns echo the information flow of a dynamic-programming parser. Nandakumar et al. (2026) supply matching results for bounded-depth, non-recursive CFGs, explicitly constructing transformers whose depth grows linearly with grammar depth and that map each nonterminal to a low-dimensional, linearly separable subspace of the residual stream. Learnability nonetheless stays fragile with depth: Li et al. (2026), probing models as in-context interpreters of novel CFGs, report that structural semantics may collapse under deep recursion or heavy branching, even while surface syntax still survives. Using invented words, they expose a reliance on the semantic priors of familiar keywords rather than genuine symbolic induction. Ghosh et al. (2026b) advocates probabilistic formal languages as semantic-contamination-free testbeds, a framework used later to pit fine-tuning against in-context learning (ICL) (Ghosh et al., 2026a). Schulz et al. (2026) decompose the language-modeling loss over a CFG’s subgrammars, showing, analytically and empirically, that, throughout training, models acquire those substructures in parallel, in contrast to human children, who learn simpler subgrammars before complex ones.

Dyck languages. Those are the languages of strings of k bracket types that are correctly balanced and nested, a canonical object of language theory and a standard probe for unbounded recursion capabilities. Hahn (2020) shows that under finite precision and Lipschitz-continuous attention a transformer decoder cannot recognize Dyck-2 on long-enough inputs. Our work does not contradict that finding, because we work on length-limited sentences, while Hahn’s impossibility is asymptotic, showing that a fixed model cannot recognize the whole of an unbounded-length Dyck-2 language.

Scaling and emergence. When and how the abilities of LLMs appear as scale grows is an open question. One school defends emergent capabilities: absent in smaller models and appearing abruptly past a critical scale, thus resisting extrapolation from the smooth scaling laws of smaller models (Wei et al., 2022a). A skeptical counter-current argues that much of the apparent suddenness is an artifact of discontinuous or nonlinear metrics (e.g., exact-match accuracy) and dissolves under continuous metrics (e.g., cross-entropy) (Schaeffer et al., 2023). A reconciling view holds that smooth scaling in pre-training loss can coexist with threshold-like jumps in downstream accuracy (Chen et al., 2025).

5 Conclusion

Our central finding is that step-by-step reasoning is not exclusive to frontier models: tiny transformers, orders of magnitude below the scale at which natural-language CoT becomes viable, already benefit from a primitive form of it. On two tasks built over the Dyck- k languages, training on a protoreasoning trace substantially closes the gap between in- and out-of-distribution validity.

Because each model is tiny and each task carries two controllable axes of difficulty, we can grade generalization exactly and train thousands of models a day on a single compute node, a scale of experimentation that would be extremely cumbersome with larger frontier models.

Expressivity and generalization show very different results: our models reach perfect in-distribution validity even for over a hundred brackets and complex structure, yet cannot zero-shot generalize to out-of-distribution samples, particularly along the structural axis. The tasks are therefore easy to fit but hard to solve in a truly algorithmic way, making them appropriate to probe reasoning at our scales.

Out-of-distribution generalization is possible, particularly for interpolation, in which case the reasoning traces beat the vanilla non-reasoning format almost always, and often by a wide margin. Moreover, the gains the reasoning traces offer come from their content, not merely from the extra tokens they introduce.

Still, the extrapolation patterns show that a truly generalizable algorithmic solution remains elusive. Without implicit or explicit signals shaping the optimization, the model has no reason to prefer solutions more robust than those the in-distribution training loss and the model selection on out-of-distribution validation enforce. The absence of cross-transfer on mixed-task training further suggests that protoreasoning does not spontaneously become compositional or modular. Richer task families and trace formats that could stimulate such compositionality are a good focus for future work. We hope the framework itself — cheap, fully controllable, and exactly gradable — will prove as useful for the community’s explorations as it was for the investigations of this paper.

References

- Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 1, learning hierarchical language structures. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=mPQKyzkA1K>.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefer. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- Yangyi Chen, Binxuan Huang, Yifan Gao, Zhengyang Wang, Jingfeng Yang, and Heng Ji. Scaling laws for predicting downstream performance in LLMs. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL <https://openreview.net/forum?id=PJUbMDkQVY>.
- David Chiang, Peter Cholak, and Anand Pillay. Tighter bounds on the expressivity of transformer encoders. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (eds.), *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 5544–5562. PMLR, 7 2023. URL <https://proceedings.mlr.press/v202/chiang23a.html>.
- Zheng Chu, Jingchang Chen, Qianglong Chen, Weijiang Yu, Tao He, Haotian Wang, Weihua Peng, Ming Liu, Bing Qin, and Ting Liu. Navigate through enigmatic labyrinth: A survey of chain of thought reasoning: Advances, frontiers and future. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1173–1203. Association for Computational Linguistics, August 2024. doi: 10.18653/v1/2024.acl-long.65. URL <https://aclanthology.org/2024.acl-long.65/>.
- Bishwamittra Ghosh, Soumi Das, Till Speicher, Qinyuan Wu, Mohammad Aflah Khan, Deepak Garg, Krishna P. Gummadi, and Evimaria Terzi. Fine-tuning vs. in-context learning in large language models: A formal language learning perspective. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 41651–41697. Association for Computational Linguistics, 2026a. URL <https://aclanthology.org/2026.acl-long.1932/>.
- Bishwamittra Ghosh, Krishna P. Gummadi, and Evimaria Terzi. A formal language benchmark for LLMs. In *3rd DATA-FM Workshop at the International Conference on Learning Representations (ICLR)*, 2026b.
- Michael Hahn. Theoretical limitations of self-attention in neural sequence models. *Transactions of the Association for Computational Linguistics*, 8:156–171, 01 2020. ISSN 2307-387X. doi: 10.1162/tacl_a_00306. URL https://doi.org/10.1162/tacl_a_00306.
- Jie Huang and Kevin Chen-Chuan Chang. Towards reasoning in large language models: A survey. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (eds.), *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 1049–1065, Toronto, Canada, 7 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.67. URL <https://aclanthology.org/2023.findings-acl.67/>.

-
- Sheraz Khan, Subha Madhavan, and Kannan Natarajan. A comment on "the illusion of thinking": Reframing the reasoning cliff as an agentic gap. arXiv:2506.18957, 6 2025.
- Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, et al. Measuring faithfulness in chain-of-thought reasoning. arXiv:2307.13702, 2023.
- Alex Lawsen. The illusion of the illusion of thinking: A comment on Shojaee et al. (2025). 2506.09250, 6 2025.
- Hanqi Li, Lu Chen, and Kai Yu. Diagnosing cfg interpretation in llms. arXiv:2604.20811, 2026. URL <https://arxiv.org/abs/2604.20811>.
- Chu-Cheng Lin, Aaron Jaech, Xin Li, Matthew R. Gormley, and Jason Eisner. Limitations of autoregressive models and their alternatives. In Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (eds.), *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 5147–5173, Online, 6 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.405. URL <https://aclanthology.org/2021.naacl-main.405/>.
- William Merrill and Ashish Sabharwal. A logic for expressing log-precision transformers. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA, 2023. Curran Associates Inc.
- Philipp Mondorf and Barbara Plank. Beyond accuracy: Evaluating the reasoning behavior of large language models – a survey. In *Proceedings of the First Conference on Language Modeling*, Philadelphia, USA, 7 2024. URL <https://openreview.net/forum?id=Lmjgl2n11u>. COLM 2024.
- Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=9XFSbDPmdW>.
- Vinoth Nandakumar, Qiang Qu, Pramod Thebe, Sakshi Khachariyapatel, and Tongliang Liu. An expressivity analysis of hierarchical modelling in deep transformers via bounded-depth grammars. arXiv:2606.17522, 2026.
- Yaniv Nikankin, Anja Reusch, Aaron Mueller, and Yonatan Belinkov. Arithmetic without algorithms: Language models solve math with a bag of heuristics. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/8c5f30296296d2ae402ebbd09aaa9c12-Abstract-Conference.html.
- Jacob Pfau, William Merrill, and Samuel R. Bowman. Let’s think dot by dot: Hidden computation in transformer language models. In *Proceedings of the 2024 Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=NikbrdtYvG>.
- Aske Plaat, Annie Wong, Suzan Verberne, Joost Broekens, Bas van Stein, and Thomas Bäck. Multi-step reasoning with large language models, a survey. *ACM Computing Surveys*, 58(6):1–35, December 2025. doi: 10.1145/3774896. URL <https://dl.acm.org/doi/10.1145/3774896>.
- Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems*, volume 36, pp. 55565–55581, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/ad98a266f45005c403b8311ca7e8bd7-Abstract-Conference.html.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

-
- Laura Ying Schulz, Daniel Mitropolsky, and Tomaso Poggio. Unraveling syntax: Language modeling and the substructure of grammars. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=sPIXjSkDFG>.
- Parshin Shojaee, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity. arXiv:2506.06941, 6 2025.
- Lena Strobl, William Merrill, Gail Weiss, David Chiang, and Dana Angluin. What formal languages can transformers express? a survey. *Transactions of the Association for Computational Linguistics*, 12:543–561, 05 2024. ISSN 2307-387X. doi: 10.1162/tacl_a_00663. URL https://doi.org/10.1162/tacl_a_00663.
- Jiankai Sun, Chuanyang Zheng, Enze Xie, Zhengying Liu, Ruihang Chu, Jianing Qiu, Jiaqi Xu, Mingyu Ding, Hongyang Li, Mengzhe Geng, Yue Wu, Wenhai Wang, Junsong Chen, Zhangyue Yin, Xiaozhe Ren, Jie Fu, Junxian He, Wu Yuan, Qi Liu, Xihui Liu, Yu Li, Hao Dong, Yu Cheng, Ming Zhang, Pheng-Ann Heng, Jifeng Dai, Ping Luo, Jingdong Wang, Ji-Rong Wen, Xipeng Qiu, Yike Guo, Hui Xiong, Qun Liu, and Zhenguo Li. A survey of reasoning with foundation models: Concepts, methodologies, and outlook. *ACM Comput. Surv.*, 57(11):278:1–278:43, 3 2025. doi: 10.1145/3729218. URL <https://doi.org/10.1145/3729218>.
- Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Keyon Vafa, Justin Y. Chen, Ashesh Rambachan, Jon Kleinberg, and Sendhil Mullainathan. Evaluating the world model implicit in a generative model. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 26941–26975. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/2f6a6317bada76b26a4f61bb70a7db59-Paper-Conference.pdf.
- Iñaki Dellibarda Varela, Pablo Romero-Sorozabal, Eduardo Rocon, and Manuel Cebrian. Rethinking the illusion of thinking. arXiv:2507.01231, 7 2025.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022a. ISSN 2835-8856. URL <https://openreview.net/forum?id=yzkSU5zdwD>. Survey Certification.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022b.
- Wenhao Wu, Yizhong Wang, Yao Fu, Xiang Yue, Dawei Zhu, and Sujian Li. Long context alignment with short instructions and synthesized positions. *arXiv preprint arXiv:2405.03939*, 2024. URL <https://arxiv.org/abs/2405.03939>.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023b.

Duzhen Zhang, Zhong-Zhi Li, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Xiuyi Chen, Yingying Zhang, Fei Yin, Jiahua Dong, Zhijiang Guo, Le Song, and Cheng-Lin Liu. From system 1 to system 2: A survey of reasoning large language models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 1–20, 2025. doi: 10.1109/TPAMI.2025.3637037. URL <https://doi.org/10.1109/TPAMI.2025.3637037>.

Dawei Zhu, Nan Yang, Liang Wang, Yifan Song, Wenhao Wu, Furu Wei, and Sujian Li. PoSE: Efficient context window extension of LLMs via positional skip-wise training. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=3Z1gxuAQRa>.

A Appendix

A.1 Holdout patterns

Fig. 7 shows precisely how we split each task’s strata into training, validation, and test sets for single-task training (Sec. 3.2). The top x-axis shows the $\log_{10}$ count of unique bracket nestings in each stratum. We select exactly 24 strata for each task, covering the region with sufficient complexity. Note that the actual number of sentences in each stratum is much larger because we have $k = 4$ options for each bracket.

Mixed-task training (Sec. 3.3) follows patterns inspired by the above, but selects them among joint strata defined by both criteria (depth and maximum preleaf). Only strata with sufficient structural diversity (at least 10^4 different bracket nestings) are considered, giving the selected region its triangular shape.

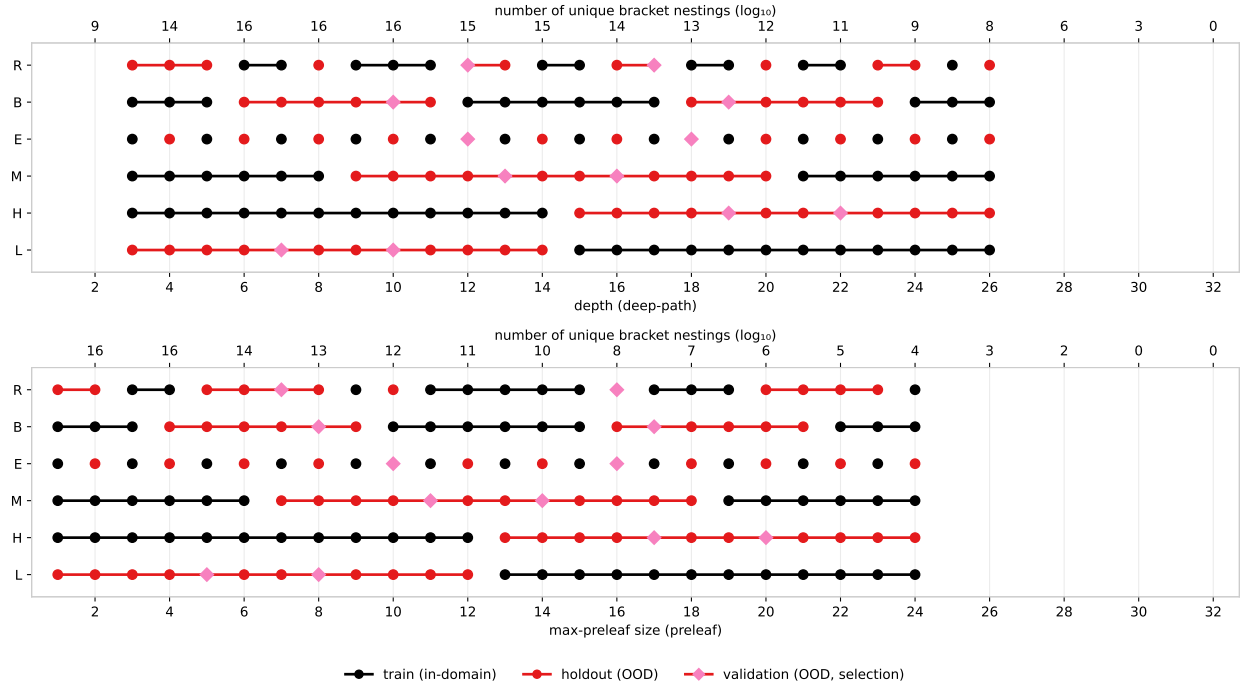


Figure 7: The six hold-out patterns (**R**andom, **B**locks, **E**ven, **M**iddle, **H**igh, and **L**ow) for single-task training (top: deepest path/DP; bottom: maximum-order preleaf/PL), with each stratum (x-axis) colored as train (in-domain), holdout (OOD), or validation. **R/B/E/M** interleave seen and held-out strata (interpolation); **H/L** hold out extreme strata (extrapolation). The study covers depths and preleaf sizes with sufficient structural variation ($\log_{10}$ count of unique bracket nestings on the top x-axes).

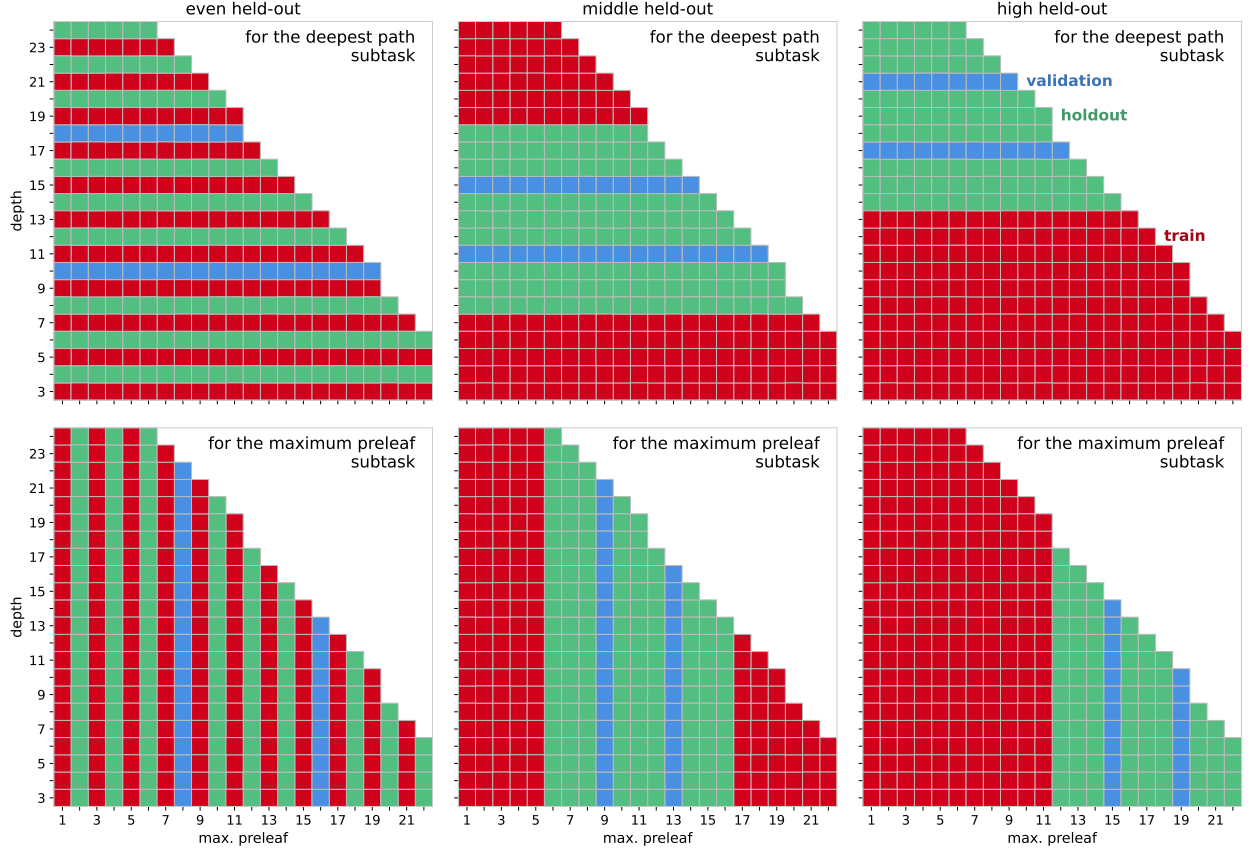


Figure 8: The three hold-out patterns (**E**ven-, **M**iddle-, and **H**igh-Mixed) for training by mixing both tasks. Each square is a joint stratum on both criteria (depth and maximum preleaf size). For a given pattern (columns), each row shows how the examples of a task (rows) are sampled. The model ends up exposed to all strata, but each task sees only its corresponding training strata.

A.2 Reasoning-trace validity

Tab. 5 shows the detailed validity results for the trace format across the two tasks and six patterns. We track validity considering: output-only (output is valid, but trace is invalid), trace-only (trace is valid, but output is invalid), both (output and trace valid), and neither (output and trace invalid). In general, trace validity follows output validity, although some samples have correct output even with an invalid trace. The converse (valid trace with invalid output) is much rarer.

A.3 Positional augmentation and step dropout

SkipAlign. To train length generalization without training on long sequences, we apply SkipAlign (Wu et al., 2024), a refinement of PoSE (Zhu et al., 2024) that remaps the position indices of a short sequence so that they span a much larger range. We insert a positional gap before the token that *starts* each reasoning step — the token immediately after a newline inside the `<BOT>...<EOT>` region, plus one gap at the start of reasoning — so the gaps land on semantic step boundaries rather than arbitrary positions. For vanilla samples, there is a single gap opportunity, between the task separator and the output.

Our SkipAlign implementation samples a target length from a log-uniform distribution, ranging from the natural length of the sample (without gaps) to the maximum possible length for a sample across all strata of that task and format. We then sample how to split the gap budget across all boundaries from a sparse Dirichlet ($\alpha=0.4$). The augmentation is a pure remapping of position indices: the tokens themselves are

Table 5: Output and trace metrics for trace-format experiments on the six single-task patterns of Fig. 7. Columns show the fraction of valid generations that are valid only on Output, only on Trace, on both parts, or on neither. Discounting numerical rounding, the four metrics sum to 100%. For the large majority of samples, output and trace validity match, although some samples have correct output despite an invalid trace. The converse is much rarer.

PATTERN	DEEPEST PATH				MAX. PRELEAF			
	OUTPUT	TRACE	BOTH	NEITHER	OUTPUT	TRACE	BOTH	NEITHER
Random	5.7 ± 1.1	0.1 ± 0.1	89.7 ± 2.7	4.6 ± 2.1	2.3 ± 0.3	0.3 ± 0.5	95.5 ± 0.6	1.9 ± 0.4
Blocks	6.0 ± 1.4	0.0 ± 0.0	92.6 ± 1.7	1.3 ± 0.4	2.7 ± 0.9	0.1 ± 0.1	94.6 ± 1.0	2.6 ± 0.5
Even	4.1 ± 2.1	0.0 ± 0.0	94.7 ± 2.9	1.2 ± 0.9	3.5 ± 0.7	0.1 ± 0.1	94.1 ± 1.1	2.2 ± 0.5
Middle	6.7 ± 6.8	0.2 ± 0.2	91.3 ± 8.8	1.9 ± 2.0	4.4 ± 0.7	1.4 ± 0.8	92.7 ± 1.1	1.5 ± 0.5
High	12.4 ± 4.4	0.2 ± 0.2	54.3 ± 8.8	33.2 ± 9.4	5.1 ± 5.4	17.0 ± 7.2	57.1 ± 9.4	20.8 ± 8.4
Low	13.0 ± 4.1	0.3 ± 0.4	8.6 ± 7.8	78.1 ± 7.6	27.4 ± 4.8	1.3 ± 1.2	13.6 ± 7.9	57.7 ± 6.3

unchanged. It encourages the model to consider positions spanning the entire range of possible input lengths, without requiring overly long contexts.

Step dropout. For trace models, step dropout independently drops each reasoning step with probability $p = 0.15$; it only deletes steps, never reorders or invents them, and leaves the final output unchanged. This forces the model to recover the answer from an arbitrary subsequence of the reasoning steps instead of memorizing a fixed-length scaffold. We score trace validity accordingly: a generated trace is valid iff its steps form an order-preserving subsequence of the reference trace’s steps (each retained step is still matched token-for-token).

A.4 Implementation Details

Model. All experiments use a Llama 2-like decoder-only transformer with rotary positional embeddings, multi-head attention, grouped-query attention disabled, and RMSNorm. Except for Sec. A.5, all experiments set width $w=128$, depth $L=4$, and 16 query/key/value heads, resulting in $\sim 1\text{M}$ parameters, orders of magnitude below even first-generation transformers.

Hyperparameters. Each treatment’s AdamW parameters (learning rate and weight decay) come from an 8-trial quasi-random (Sobol) search selected according to output validity on the validation strata, which are disjoint from both training and test (Sec. A.1). The expressivity studies, which train on a single stratum, select hyperparameters in-domain. The search space for the parameters has learning rate from 10^{-4} to 10^{-2} and weight decay from 10^{-7} to 10^{-1} . Fixed parameters are AdamW’s $\beta_1 = 0.9$, $\beta_2 = 0.95$, and $\epsilon = 10^{-6.5}$.

Training length and schedule. We train all models with a batch size of 256 for 6K steps, with a warmup–stable–decay learning-rate schedule (600 warmup and 600 decay steps). We pretrain each model for 2K steps, during which the autoregressive loss applies to the entire sample, and then fine-tune it for 4K steps, with the loss accounting only for the generated part (including the trace, if present). We apply SkipAlign for 1.3K steps at the beginning of the fine-tuning phase.

Evaluation. Unless noted otherwise, we report greedy-decoding exact-match accuracy at the best-validation checkpoint. Evaluation is over the holdout strata (out of domain) for the generalization experiments and over the training strata (in domain) for the expressivity experiments. We score output validity on the validation strata every 500 training steps and evaluate the checkpoint that maximizes it, breaking ties toward the earliest step; the same best-checkpoint criterion scores each trial of the hyperparameter search.

For trace models, selection always ranks on the output-only validity. All validation is over a fixed set of 128 prompts per stratum.

Compute. The full study comprises 186 treatments trained over 10 seeds (1860 final training runs plus 1488 hyperparameter-search training runs). The complete study takes a little over a day of wall-clock time on a single multi-GPU node with $7\times$ NVIDIA B200 GPUs.

A.5 Model and language scaling

In this section, we measure the ability of different small models to express and generalize both tasks on languages of different lengths, all for the vanilla data format. For each language, we split the strata randomly as training, validation, and test (the Random pattern of Sec. A.1). As usual, we measure expressivity on different in-domain sentences sampled from the training strata and generalization on out-of-domain sentences sampled from the test strata.

Fig. 9 shows that, among very small models, there is a relatively sharp frontier between those that can express the task for a certain length and those that cannot. It also shows that generalization is at least as hard as expressivity. For the bulk of this work, we used half-length = 32 and a model with residual width = 128, 4 attention layers, and 16 Q/K/V attention heads, a scenario where expressivity is near-perfect but generalization already poses some challenge, even for the relatively easy Random pattern.

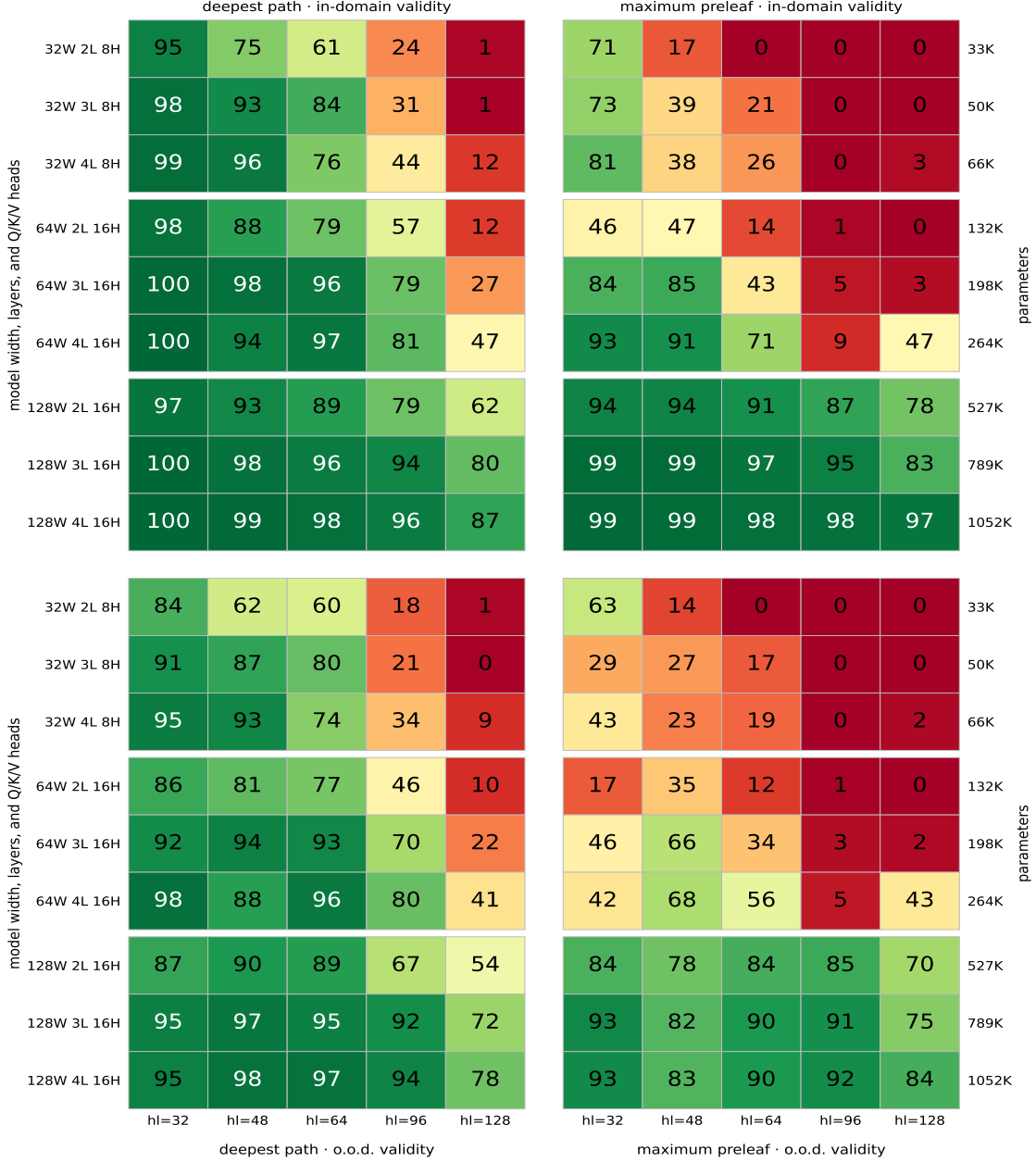


Figure 9: In-domain expressivity (top row) and out-of-domain generalization for models trained and tested on randomly split structural strata. Model configuration appears on the left axis (width, number of layers, number of Q/K/V heads), parameter count on the right axis. The bottom axis shows the half length of the language parameters used to train and evaluate the model.